

SEG3503 Lab 1
Nina Faith Ukoha
nukoh033@uottawa.ca

Git et Github

Lien vers github: https://github.com/nina033/seg3503_playground/

Java version:

```
ninaukoha@Ninas-MacBook-Air-2 newmath_ex % java --version
openjdk 17.0.12 2024-07-16
OpenJDK Runtime Environment JBR-17.0.12+1-1207.37-nomod (build 17.0.12+1-b1207.37)
OpenJDK 64-Bit Server VM JBR-17.0.12+1-1207.37-nomod (build 17.0.12+1-b1207.37, mixed mode, sharing)
```

Elixir version:

```
ninaukoha@Ninas-MacBook-Air-2 newmath_ex % elixir --version
Erlang/OTP 29 [erts-17.0.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Elixir 1.19.5 (compiled with Erlang/OTP 29)
```

Java run:

```
ninaukoha@Ninas-MacBook-Air-2 newmath_java % ./bin/run
Newmath (type 'exit' to exit program)
Numerator: 10
Demoninator: 5
10 / 5 = 2
Numerator: 20
Demoninator: 3
20 / 3 = 6
Numerator: exit
```

Java test

```
ninaukoha@Ninas-MacBook-Air-2 newmath_java % ./bin/test

Thanks for using JUnit! Support its development at https://junit.org/sponsoring

└─ JUnit Jupiter ✓
  └─ NewmathTest ✓
    └─ div_ok() ✓
      └─ div_by_zero() ✓
└─ JUnit Vintage ✓

Test run finished after 29 ms
[      3 containers found      ]
[      0 containers skipped    ]
[      3 containers started    ]
[      0 containers aborted    ]
[      3 containers successful ]
[      0 containers failed     ]
[      2 tests found          ]
[      0 tests skipped         ]
[      2 tests started         ]
[      0 tests aborted         ]
[      2 tests successful      ]
[      0 tests failed          ]
```

Elixir run

```
ninaukoha@Ninas-MacBook-Air-2 newmath_ex % ./bin/run
Erlang/OTP 29 [erts-17.0.1] [source] [64-bit] [smp:8:8] [ds:8:8:10] [async-threads:1] [jit] [dtrace]

Interactive Elixir (1.19.5) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> NewmathEx.div(5,2)
{:ok, 2.5}
iex(2)> NewmathEx.div(5,0)
{:error, "Cannot divide by zero"}
iex(3)> 
```

Elixir test

```
ninaukoha@Ninas-MacBook-Air-2 newmath_ex % ./bin/test
Running ExUnit with seed: 536651, max_cases: 16
```

```
  warning: comparison between distinct types found:
```

```
    left == right
```

```
given types:
```

```
dynamic({:error, binary()}) or {:ok, float()}) == {:ok, integer()}
```

```
where "left" (context ExUnit.Assertions) was given the type:
```

```
  # type: dynamic({:error, binary()}) or {:ok, float()})
```

```
  # from: test/newmath_ex_test.exs:7
```

```
  left = NewmathEx.div(4, 2)
```

```
where "right" (context ExUnit.Assertions) was given the type:
```

```
  # type: {:ok, integer()}
```

```
  # from: test/newmath_ex_test.exs:7
```

```
  right = {:ok, 2}
```

While Elixir can compare across all types, you are comparing across types which are always disjoint, and the result is either always true or always false

```
typing violation found at:
```

```
7 |   assert NewmathEx.div(4, 2) == {:ok, 2}
```

```
   |   test/newmath_ex_test.exs:7:34: NewmathExTest."test div/1 ok"/1
```

```
***
Finished in 0.02 seconds (0.00s async, 0.02s sync)
```

```
1 doctest, 2 tests, 0 failures
```